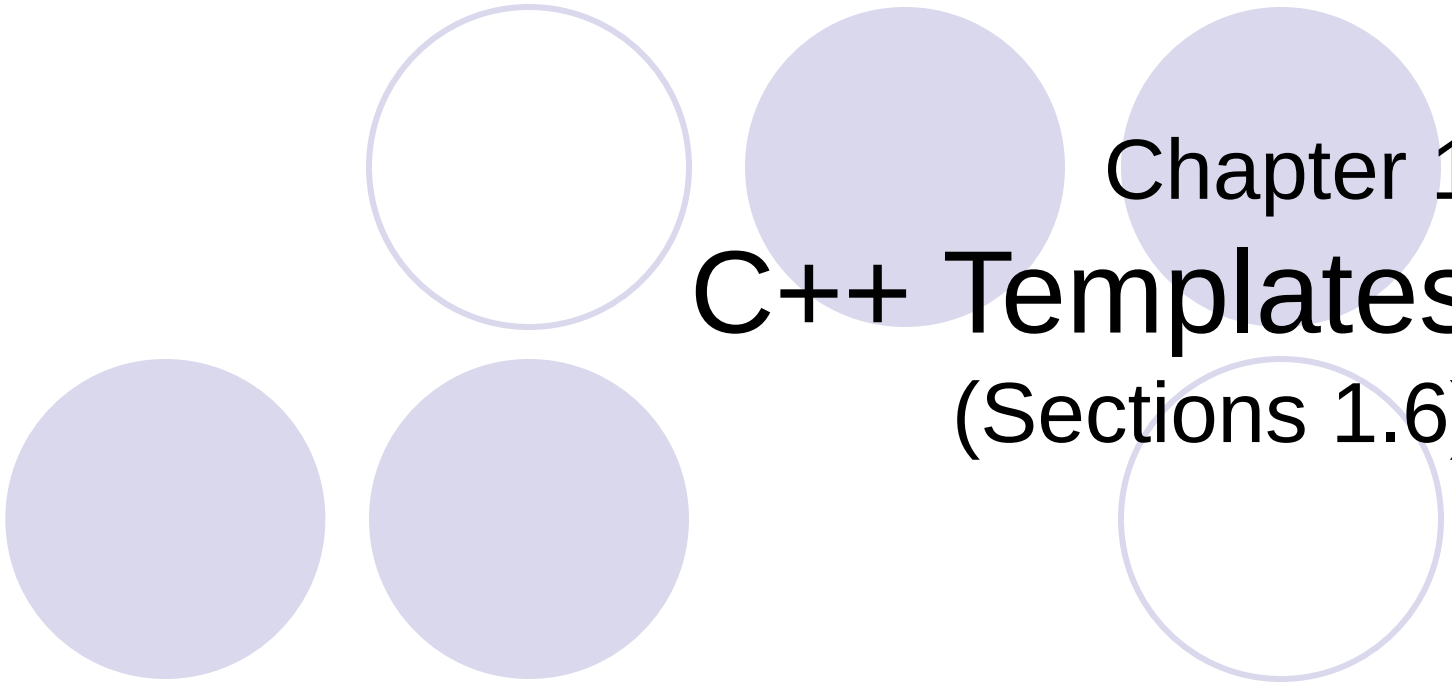


Chapter 1
C++ Templates
(Sections 1.6)

Templates

- Type-independent patterns that can work with multiple data types.
- Function Templates
  - These define logic behind the algorithms that work for multiple data types.
- Class Templates
  - These define generic class patterns into which specific data types can be plugged in to produce new classes.

Function Templates example

- Generic function to find a maximum value in a given vector
- If argument is a `vector<int>` type, then compiler generates a corresponding function which Comparable is replaced by `vector<int>` type.
- Similarly for `vector<double>`, `vector<string>` etc.
- Assumption in this example:
  - `Operator<` is defined in the Comparable.
- Hence assumptions made by the template much be clearly stated.

```
1  /**
2   * Return the maximum item in array a.
3   * Assumes a.size( ) > 0.
4   * Comparable objects must provide operator< and operator=
5   */
6  template <typename Comparable>
7  const Comparable & findMax( const vector<Comparable> & a )
8  {
9      int maxIndex = 0;
10
11     for( int i = 1; i < a.size( ); i++ )
12         if( a[ maxIndex ] < a[ i ] )
13             maxIndex = i;
14     return a[ maxIndex ];
15 }
```

Function Templates Usage

- Each call to `findMax(Comparable a)` on a different data type forces the compiler to generate a different function using the template.
- Compiler will complain about `findMax(v4)` because IntCell class does not define `operator<`

```
1  int main( )
2  {
3      vector<int>    v1( 37 );
4      vector<double> v2( 40 );
5      vector<string> v3( 80 );
6      vector<IntCell> v4( 75 );
7
8      // Additional code to fill in the vectors not shown
9
10     cout << findMax( v1 ) << endl;  // OK: Comparable = int
11     cout << findMax( v2 ) << endl;  // OK: Comparable = double
12     cout << findMax( v3 ) << endl;  // OK: Comparable = string
13     cout << findMax( v4 ) << endl;  // Illegal; operator< undefined
14
15     return 0;
16 }
```

Class Template Example

- `MemoryCell` template can be used for any type `Object`.
- Assumptions
  - `Object` has a zero parameter constructor
  - `Object` has a copy constructor
  - Copy-assignment operator
- Convention
  - Class templates declaration and implementation usually combined in a single file.
  - It is not easy to separate them in independent files due to complex C++ syntax.
  - This is different from the convention of separating class interface and implementation in different files.

```
1  /**
2   * A class for simulating a memory cell.
3   */
4  template <typename Object>
5  class MemoryCell
6  {
7    public:
8      explicit MemoryCell( const Object & initialValue = Object{ } )
9        : storedValue{ initialValue } { }
10     const Object & read( ) const
11       { return storedValue; }
12     void write( const Object & x )
13       { storedValue = x; }
14   private:
15     Object storedValue;
16 };
```

Class Template Usage Example

- `MemoryCell` can be used to store both primitive and class types.

```
1   int main( )
2   {
3       MemoryCell<int>    m1;
4       MemoryCell<string> m2{ "hello" };
5
6       m1.write( 37 );
7       m2.write( m2.read( ) + "world" );
8       cout << m1.read( ) << endl << m2.read( ) << endl;
9
10      return 0;
11  }
```

- Remember
  - `MemoryCell` is not a class.
  - It's a class template.
  - `MemoryCell<int>`, `MemoryCell<string>` are classes.

Operator Overloading

- It allows C++ operators to be used with user-defined types (such as classes).
- Use the keyword (operator) followed by the operator symbol (ex. operator+).

ex. : The statement (cout<<a+b;) will generate an error unless an operator+ is exists

```
class Complex{};// any class
main()
{Complex a;
Complex b;
cout<<a+b; // error.. + is not used with class data types}
```

Operator Overloading

- The operator function can be a member function or a friend function.

- Member function:

```
class Complex { public:
int real;
Complex(int a=int{})
: (real=a) {}
int operator + (const Complex& obj)
{ return real+obj.real;}
};
```

Operator Overloading

- friend function:

```
class Complex { public:
int real;
Complex(int a=int{})
: (real=a) {}
friend int operator + (const Complex& obj1, const Complex& obj2 );
};
int (operator + const Complex& obj1, const Complex& obj2 )
{return obj1.real+obj2.real;}
void main()
{Complex a(10);
Complex b(20);
cout<<a+b;}
```

Matrices

- C++ library does not provide a **`matrix`** class
- Constructor
  - Creates `rows` number of zero-sized vectors
  - Resizes each vector to `col` elements
- Two types of `[]` operators
  - One for LHS that returns by reference
  - Another for RHS that returns by constant reference
  - So we have two very identical functions
    - What makes their signatures different?

```
1   #ifndef MATRIX_H
2   #define MATRIX_H
3
4   #include <vector>
5   using namespace std;
6
7   template <typename Object>
8   class matrix
9   {
10    public:
11      matrix( int rows, int cols ) : array( rows )
12      {
13          for( int i = 0; i < rows; i++ )
14              array[ i ].resize( cols );
15      }
16
17      const vector<Object> & operator[]( int row ) const
18        { return array[ row ]; }
19      vector<Object> & operator[]( int row )
20        { return array[ row ]; }
21
22      int numrows( ) const
23        { return array.size( ); }
24      int numcols( ) const
25        { return numrows( ) ? array[ 0 ].size( ) : 0; }
26    private:
27      vector< vector<Object> > array;
28  };
29
30  #endif
```